

MD_Notebook_Matthew_Choo
_ITAI_2376
L08 Diffusion Models

Creating Numbers/Images with AI: A Hands-on Diffusion Model Exercise

Introduction

In this assignment, you'll learn how to create an AI model that can generate realistic images from scratch using a powerful technique called 'diffusion'. Think of it like teaching AI to draw by first learning how images get blurry and then learning to make them clear again.

What We'll Build

- A diffusion model capable of generating realistic images
- For most students: An AI that generates handwritten digits (0-9) using the MNIST dataset
- For students with more computational resources: Options to work with more complex datasets
- Visual demonstrations of how random noise gradually transforms into clear, recognizable images
- By the end, your AI should create images realistic enough for another AI to recognize them

Dataset Options

This lab offers flexibility based on your available computational resources:

- Standard Option (Free Colab):** We'll primarily use the MNIST handwritten digit dataset, which works well with limited GPU memory and completes training in a reasonable time frame. Most examples and code in this notebook are optimized for MNIST.
- Advanced Option:** If you have access to more powerful GPUs (either through Colab Pro/Pro+ or your own hardware), you can experiment with more complex datasets like Fashion-MNIST, CIFAR-10, or even face generation. You'll need to adapt the model architecture, hyperparameters, and evaluation metrics accordingly.

Resource Requirements

- Basic MNIST:** Works with free Colab GPUs (2-4GB VRAM) - 30 minutes training
- Fashion-MNIST:** Similar requirements to MNIST/CIFAR-10. Requires more memory (8-12GB VRAM) and longer training (~2 hours)
- Higher resolution images:** Requires substantial GPU resources and several hours of training

Before You Start

- Make sure you're running this in Google Colab or another environment with GPU access
- Go to Runtime → Change runtime type and select GPU as your hardware accelerator
- Each code cell has comments explaining what it does
- Don't worry if you don't understand every detail - focus on the big picture!
- If working with larger datasets, monitor your GPU memory usage carefully

The concepts you learn with MNIST will scale to more complex datasets, so even if you're using the basic option, you'll gain valuable knowledge about generative AI that applies to more advanced applications.

Step 1: Setting Up Our Tools

First, let's install and import all the tools we need. Run this cell and wait for it to complete.

```
# Step 1: Install required packages
!pip install einops
print("Package installation complete.")

# Step 2: Import libraries
# --- Core PyTorch libraries ---
import torch # Main deep learning framework
import torch.nn.functional as F # Neural network functions like activation functions
import torch.nn as nn # Neural network building blocks (layers)
from torch.optim import Adam # Optimization algorithm for training

# --- Data handling ---
from torch.utils.data import Dataset, DataLoader # For organizing and loading our data
import torchvision # Library for computer vision datasets and models
import torchvision.transforms as transforms # For preprocessing images

# --- Tensor manipulation ---
import random # For random operations
from einops.layers.torch import Rearrange # For reshaping tensors in neural networks
from einops import rearrange # For elegant tensor reshaping operations
import numpy as np # For numerical operations on arrays

# --- System utilities ---
import os # For operating system interactions (used for CPU count)

# --- Visualization tools ---
import matplotlib.pyplot as plt # For plotting images and graphs
from PIL import Image # For image processing
from torchvision.utils import save_image, make_grid # For saving and displaying image grids

# Step 3: Set up device (GPU or CPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"We'll be using: {device}")

# Check if we're actually using GPU (for students to verify)
if device.type == "cuda":
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
    print(f"GPU memory: {torch.cuda.get_device_properties(0).total_memory / 1024**2} GB")
else:
    print("Note: Training will be much slower on CPU. Consider using Google Colab with GPU enabled.")

--- Requirement already satisfied: einops in /usr/local/lib/python3.12/dist-packages (0.6.2)
Package installation complete.
We'll be using: CPU
Note: Training will be much slower on CPU. Consider using Google Colab with GPU enabled.
```

REPRODUCIBILITY AND DEVICE SETUP

```
# Step 4: Set random seeds for reproducibility
# Diffusion models are sensitive to initialization, so reproducible results help with debugging
SEED = 42 # Universal seed value for reproducibility
torch.manual_seed(SEED) # PyTorch random number generator
np.random.seed(SEED) # Numpy random number generator
random.seed(SEED) # Python's built-in random number generator

print(f"Random seeds set to {SEED} for reproducible results")

# Configure CUDA for GPU operations if available
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED) # GPU random number generator
    torch.cuda.manual_seed_all(SEED) # All GPU random number generator

# Ensure deterministic GPU operations
# Note: This slightly reduces performance but ensures results are reproducible
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

try:
    # Check available GPU memory
    gpu_memory = torch.cuda.get_device_properties(0).total_memory / 1024 # Convert to GB
    print(f"Available GPU memory: {gpu_memory:.1f} GB")

    # Add recommendation based on memory
    if gpu_memory < 4:
        print("Warning: Low GPU memory. Consider reducing batch size if you encounter OOM errors.")
    except Exception as e:
        print("Could not check GPU memory: {e}")
    else:
        print("No GPU detected. Training will be much slower on CPU.")
        print("If you're using Colab, go to Runtime > Change runtime type and select GPU.")

--- Random seeds set to 42 for reproducible results
No GPU detected. Training will be much slower on CPU.
If you're using Colab, go to Runtime > Change runtime type and select GPU.
```

✓ Step 2: Choosing Your Dataset

You have several options for this exercise, depending on your computer's capabilities:

Option 1: MNIST (Basic - Works on Free Colab)

- Content: Handwritten digits (0-9)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if:** You're using free Colab or have a basic GPU

Option 2: Fashion-MNIST (Intermediate)

- Content: Clothing items (shirts, shoes, etc.)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if:** You want more interesting images but have limited GPU

Option 3: CIFAR-10 (Advanced)

- Content: Real-world objects (cars, animals, etc.)
- Image size: 32x32 pixels, Color (RGB)
- Training samples: 50,000
- Memory needed: ~4GB GPU
- Training time: ~1-2 hours on Colab
- **Choose this if:** You have Colab Pro or a good local GPU (8GB+ memory)

Option 4: CelebA (Expert)

- Content: Celebrity face images
- Image size: 64x64 pixels, Color (RGB)
- Training samples: 200,000
- Memory needed: ~8GB GPU
- Training time: ~3-4 hours on Colab
- **Choose this if:** You have excellent GPU (12GB+ memory)

To use your chosen dataset, uncomment its section in the code below and make sure all others are commented out.

```

▶ IMG_SIZE = 28
  IMG_CH = 1
  N_CLASSES = 10
  BATCH_SIZE = 64
  EPOCHS = 30

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Your code to load the MNIST dataset
# Hint: Use torchvision.datasets.MNIST with root='./data', train=True,
#       transform=transform, and download=True
# Then print a success message

# Enter your code here:
dataset = torchvision.datasets.MNIST(root='./data', train=True,
                                     transform=transform, download=True)
print("MNIST dataset loaded successfully!")

#-----
# OPTION 2: Fashion-MNIST (Intermediate - 2GB GPU)
#-----
# Uncomment this section to use Fashion-MNIST instead
***

IMG_SIZE = 28
IMG_CH = 1
N_CLASSES = 10
BATCH_SIZE = 64
EPOCHS = 30

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Your code to load the Fashion-MNIST dataset
# Hint: Very similar to MNIST but use torchvision.datasets.FashionMNIST

# Enter your code here:

***

#-----
# OPTION 3: CIFAR-10 (Advanced - 4GB+ GPU)
#-----
# Uncomment this section to use CIFAR-10 instead
***

```

```

IMG_SIZE = 32
IMG_CH = 3
N_CLASSES = 10
BATCH_SIZE = 32 # Reduced batch size for memory
EPOCHS = 50 # More epochs for complex data

# Your code to create the transform and load CIFAR-10
# Hint: Use transforms.Normalize with RGB means and stds ((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
# Then load torchvision.datasets.CIFAR10

# Enter your code here:
***

```

```

... 100% 0.55M/0.55M [00:00<00:00, 63.1M/s]
100% 10.9M/10.9M [00:00<00:00, 1.39M/s]
100% 1.69M/1.69M [00:00<00:00, 14.9M/s]
100% 4.54M/4.54M [00:00<00:00, 4.99M/s] MNIST dataset loaded successfully!

```

^100% SIZE = 32100M CH = 3100M CLASSES = 10100M BATCH_SIZE = 32 # Reduced batch size for memory/EPOCHS = 50 # More epochs for complex data/In your code to create the transform and load CIFAR-10 Hint: Use transforms.Normalize with RGB means and stds ((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))/In Then load torchvision.datasets.CIFAR10/In Enter your code here:/In/

```

#Validating Dataset Selection
#Let's add code to validate that a dataset was selected
# and check if your GPU has enough memory:

# Validate dataset selection
if 'dataset' not in locals():
    raise ValueError("""
    ✗ ERROR: No dataset selected! Please uncomment exactly one dataset option.
    Available options:
    1. MNIST (Basic) - 2GB GPU
    2. Fashion-MNIST (Intermediate) - 2GB GPU
    3. CIFAR-10 (Advanced) - 4GB+ GPU
    4. CelebA (Expert) - 8GB+ GPU
    """)

# Your code to validate GPU memory requirements
# Hint: Check torch.cuda.is_available() and use torch.cuda.get_device_properties(0).total_memory
# to get available GPU memory, then compare with dataset requirements

# Enter your code here:
if torch.cuda.is_available():
    gpu_memory_gb = torch.cuda.get_device_properties(0).total_memory / (1024**3)
    print(f"Detected GPU with {gpu_memory_gb:.2f} GB of VRAM.")

    # Assuming MNIST (2GB) is chosen for this example
    required_memory_gb = 2 # Based on MNIST option

    if gpu_memory_gb < required_memory_gb:
        print(f"WARNING: Your GPU ({gpu_memory_gb:.2f} GB) might have insufficient memory for this dataset ((required_memory_gb) GB required). Consider reducing batch size or switching to a less demanding dataset.")
    else:
        print(f"GPU memory is sufficient ({gpu_memory_gb:.2f} GB available, (required_memory_gb) GB required).")
else:
    print("No GPU detected. Proceeding with CPU. Performance may be significantly slower.")

```

... No GPU detected. Proceeding with CPU. Performance may be significantly slower.

```

#Validating Dataset Selection
#Let's add code to validate that a dataset was selected
# and check if your GPU has enough memory:

# Validate dataset selection
if 'dataset' not in locals():
    raise ValueError("""
    ✗ ERROR: No dataset selected! Please uncomment exactly one dataset option.
    Available options:
    1. MNIST (Basic) - 2GB GPU
    2. Fashion-MNIST (Intermediate) - 2GB GPU
    3. CIFAR-10 (Advanced) - 4GB+ GPU
    4. CelebA (Expert) - 8GB+ GPU
    """)

# Your code to validate GPU memory requirements
# Hint: Check torch.cuda.is_available() and use torch.cuda.get_device_properties(0).total_memory
# to get available GPU memory, then compare with dataset requirements

# Enter your code here:
if torch.cuda.is_available():
    gpu_memory_gb = torch.cuda.get_device_properties(0).total_memory / (1024**3)
    print(f"Detected GPU with {gpu_memory_gb:.2f} GB of VRAM.")

    # Assuming MNIST (2GB) is chosen for this example
    required_memory_gb = 2 # Based on MNIST option

    if gpu_memory_gb < required_memory_gb:
        print(f"WARNING: Your GPU ({gpu_memory_gb:.2f} GB) might have insufficient memory for this dataset ((required_memory_gb) GB required). Consider reducing batch size or switching to a less demanding dataset.")
    else:
        print(f"GPU memory is sufficient ({gpu_memory_gb:.2f} GB available, (required_memory_gb) GB required).")
else:
    print("No GPU detected. Proceeding with CPU. Performance may be significantly slower.")

```

... No GPU detected. Proceeding with CPU. Performance may be significantly slower.

```

#Validating Dataset Selection
#Let's add code to validate that a dataset was selected
# and check if your GPU has enough memory:

# Validate dataset selection
if 'dataset' not in locals():
    raise ValueError("""
    ✗ ERROR: No dataset selected! Please uncomment exactly one dataset option.
    Available options:
    1. MNIST (Basic) - 2GB GPU
    2. Fashion-MNIST (Intermediate) - 2GB GPU
    3. CIFAR-10 (Advanced) - 4GB+ GPU
    4. CelebA (Expert) - 8GB+ GPU
    """)

# Your code to validate GPU memory requirements
# Hint: Check torch.cuda.is_available() and use torch.cuda.get_device_properties(0).total_memory
# to get available GPU memory, then compare with dataset requirements

# Enter your code here:
if torch.cuda.is_available():
    gpu_memory_gb = torch.cuda.get_device_properties(0).total_memory / (1024**3)
    print(f"Detected GPU with {gpu_memory_gb:.2f} GB of VRAM.")

    # Assuming MNIST (2GB) is chosen for this example
    required_memory_gb = 2 # Based on MNIST option

    if gpu_memory_gb < required_memory_gb:
        print(f"WARNING: Your GPU ({gpu_memory_gb:.2f} GB) might have insufficient memory for this dataset ((required_memory_gb) GB required). Consider reducing batch size or switching to a less demanding dataset.")
    else:
        print(f"GPU memory is sufficient ({gpu_memory_gb:.2f} GB available, (required_memory_gb) GB required).")
else:
    print("No GPU detected. Proceeding with CPU. Performance may be significantly slower.")

```

No GPU detected. Proceeding with CPU. Performance may be significantly slower.

```

#Dataset Properties and Data Loaders
#Now let's examine our dataset
#and set up the data loaders:

# Your code to check sample batch properties
# Hint: Get a sample batch using next(iter(DataLoader(dataset, batch_size=1)))
# Then print information about the dataset shape, type, and value ranges

# Enter your code here:
sample_image, sample_label = next(iter(DataLoader(dataset, batch_size=1)))
print(f"\nSample Batch Properties:")
print(f" Image shape: {sample_image.shape}") # Should be [1, IMG_CH, IMG_SIZE, IMG_SIZE]
print(f" Image data type: {sample_image.dtype}")
print(f" Image value range: [{sample_image.min():.2f}, {sample_image.max():.2f}]") # Should be between -1 and 1
print(f" Label: {sample_label.item()}")
print(f" Label data type: {sample_label.dtype}")

#=====
# SECTION 3: DATASET SPLITTING AND DATALOADER CONFIGURATION
#=====
# Create train-validation split

# Your code to create a train-validation split (80% train, 20% validation)
# Hint: Use random_split() with appropriate train_size and val_size
# Be sure to use a fixed generator for reproducibility

# Enter your code here:
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(
    dataset, [train_size, val_size], generator=torch.Generator().manual_seed(SEED)
)
print(f"\nDataset split: {len(train_dataset)} training samples, {len(val_dataset)} validation samples")

# Your code to create dataloaders for training and validation
# Hint: Use DataLoader with batch_size=BATCH_SIZE, appropriate shuffle settings,
# and num_workers based on available CPU cores

# Enter your code here:
# Determine number of workers, considering CPU execution
num_workers = min(os.cpu_count() if os.cpu_count() else 0, 4) # Cap at 4 for common Colab CPU limits

train_dataloader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=num_workers, pin_memory=True if device.type == 'cuda' else False
)
val_dataloader = DataLoader(
    val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=num_workers, pin_memory=True if device.type == 'cuda' else False
)
print(f"Dataloaders created with batch size {BATCH_SIZE} and {num_workers} workers.")

```

```

***
Sample Batch Properties:
Image shape: torch.Size([1, 1, 28, 28])
Image data type: torch.float32
Image value range: [-1.00, 1.00]
Label: 5
Label data type: torch.int64

Dataset split: 48000 training samples, 12000 validation samples
Dataloaders created with batch size 64 and 2 workers.

```

Step 3: Building Our Model Components

Now we'll create the building blocks of our AI model. Think of these like LEGO pieces that we'll put together to make our number generator:

- GELUConvBlock: The basic building block that processes images
- DownBlock: Makes images smaller while finding important features
- UpBlock: Makes images bigger again while keeping the important features
- Other blocks: Help the model understand time and what number to generate

```
class GELUConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, group_size):
        """
        Creates a block with convolution, normalization, and activation

        Args:
            in_ch (int): Number of input channels
            out_ch (int): Number of output channels
            group_size (int): Number of groups for GroupNorm
        """
        super().__init__()

        # Check that group_size is compatible with out_ch
        if out_ch % group_size != 0:
            print(f"Warning: out_ch ({out_ch}) is not divisible by group_size ({group_size})")
            # Adjust group_size to be compatible
            group_size = min(group_size, out_ch)
            while out_ch % group_size != 0:
                group_size -= 1
            print(f"Adjusted group_size to {group_size}")
            if group_size == 0: # Avoid division by zero
                group_size = 1 # Fallback to group_size 1 if no compatible group_size found

        # Your code to create layers for the block
        # Hint: Use nn.Conv2d, nn.GroupNorm, and nn.GELU activation
        # Then combine them using nn.Sequential

        # Enter your code here:
        self.model = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.GroupNorm(group_size, out_ch),
            nn.GELU()
        )

    def forward(self, x):
        # Your code for the forward pass
        # Hint: Simply pass the input through the model

        # Enter your code here:
        return self.model(x)
```

```

▶ # Rearranges pixels to downsample the image (2x reduction in spatial dimensions)
class RearrangePoolBlock(nn.Module):
    def __init__(self, in_chs, group_size):
        """
        Downsamples the spatial dimensions by 2x while preserving information

        Args:
            in_chs (int): Number of input channels
            group_size (int): Number of groups for GroupNorm
        """
        super().__init__()

        # Your code to create the rearrange operation and convolution
        # Hint: Use Rearrange from einops.layers.torch to reshape pixels
        # Then add a GELUConvBlock to process the rearranged tensor

        # Enter your code here:
        self.rearrange = Rearrange('b c (h p1) (w p2) -> b (c p1 p2) h w', p1 = 2, p2 = 2)
        self.conv = GELUConvBlock(in_chs * 4, in_chs * 4, group_size)

    def forward(self, x):
        # Your code for the forward pass
        # Hint: Apply rearrange to downsample, then apply convolution

        # Enter your code here:
        return self.conv(self.rearrange(x))

```



```

#Let's implement the upsampling block for our U-Net architecture:
class DownBlock(nn.Module):
    """
    Downsampling block for encoding path in U-Net architecture.

    This block:
    1. Processes input features with two convolutional blocks
    2. Downsamples spatial dimensions by 2x using pixel rearrangement

    Args:
        in_chs (int): Number of input channels
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
        super().__init__() # Simplified super() call, equivalent to original

        # Sequential processing of features
        layers = [
            GELUConvBlock(in_chs, out_chs, group_size), # First conv block changes channel dimensions
            GELUConvBlock(out_chs, out_chs, group_size), # Second conv block processes features
            RearrangePoolBlock(out_chs, group_size)      # Downsampling (spatial dims: H,W → H/2,W/2)
        ]
        self.model = nn.Sequential(*layers)

        # Log the configuration for debugging
        print(f"Created DownBlock: in_chs={in_chs}, out_chs={out_chs}, spatial_reduction=2x")

    def forward(self, x):
        """
        Forward pass through the DownBlock.

        Args:
            x (torch.Tensor): Input tensor of shape [B, in_chs, H, W]

        Returns:
            torch.Tensor: Output tensor of shape [B, out_chs, H/2, W/2]
        """
        return self.model(x)

```

```

#Now let's implement the upsampling block for our U-Net architecture:
class UpBlock(nn.Module):
    """
    Upsampling block for decoding path in U-Net architecture.

    This block:
    1. Takes features from the decoding path and corresponding skip connection
    2. Concatenates them along the channel dimension
    3. Upsamples spatial dimensions by 2x using transposed convolution
    4. Processes features through multiple convolutional blocks

    Args:
        in_chs (int): Number of input channels from the previous layer
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
        super().__init__()

        # Your code to create the upsampling operation
        # Hint: Use nn.ConvTranspose2d with kernel_size=2 and stride=2
        # Note that the input channels will be 2 * in_chs due to concatenation

        # Enter your code here:
        self.upsample = nn.ConvTranspose2d(in_chs * 2, in_chs, kernel_size=2, stride=2)

        # Your code to create the convolutional blocks
        # Hint: Use multiple GELUConvBlocks in sequence

        # Enter your code here:
        self.conv_blocks = nn.Sequential(
            GELUConvBlock(in_chs * 2, out_chs, group_size),
            GELUConvBlock(out_chs, out_chs, group_size)
        )

        # Log the configuration for debugging
        print(f"Created UpBlock: in_chs={in_chs}, out_chs={out_chs}, spatial_increas=2x")

    def forward(self, x, skip):
        """
        Forward pass through the UpBlock.

        Args:
            x (torch.Tensor): Input tensor from previous layer [B, in_chs, H, W]
            skip (torch.Tensor): Skip connection tensor from encoder [B, in_chs, 2H, 2W]

        Returns:
            torch.Tensor: Output tensor with shape [B, out_chs, 2H, 2W]
        """

        # Your code for the forward pass
        # Hint: Concatenate x and skip, then upsample and process

        # Enter your code here:
        x = self.upsample(x)
        x = torch.cat([x, skip], dim=1)
        return self.conv_blocks(x)

```

```

▶ # Here we implement the time embedding block for our U-Net architecture:
# Helps the model understand time steps in diffusion process
class SinusoidalPositionEmbedBlock(nn.Module):
    """
    Creates sinusoidal embeddings for time steps in diffusion process.

    This embedding scheme is adapted from the Transformer architecture and
    provides a unique representation for each time step that preserves
    relative distance information.

    Args:
        dim (int): Embedding dimension
    """
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, time):
        """
        Computes sinusoidal embeddings for given time steps.

        Args:
            time (torch.Tensor): Time steps tensor of shape [batch_size]

        Returns:
            torch.Tensor: Time embeddings of shape [batch_size, dim]
        """
        device = time.device
        half_dim = self.dim // 2
        embeddings = torch.log(torch.tensor(10000.0, device=device)) / (half_dim - 1)
        embeddings = torch.exp(torch.arange(half_dim, device=device) * -embeddings)
        embeddings = time[:, None] * embeddings[None, :]
        embeddings = torch.cat((embeddings.sin(), embeddings.cos()), dim=-1)
        return embeddings

```

```

# Helps the model understand which number/image to draw (class conditioning)
class EmbedBlock(nn.Module):
    """
    Creates embeddings for class conditioning in diffusion models.

    This module transforms a one-hot or index representation of a class
    into a rich embedding that can be added to feature maps.

    Args:
        input_dim (int): Input dimension (typically number of classes)
        emb_dim (int): Output embedding dimension
    """
    def __init__(self, input_dim, emb_dim):
        super(EmbedBlock, self).__init__()
        self.input_dim = input_dim
        self.model = nn.Sequential(
            nn.Linear(input_dim, emb_dim),
            nn.GELU(),
            nn.Linear(emb_dim, emb_dim),
            nn.Unflatten(-1, (emb_dim, 1, 1)) # Reshape for broadcasting
        )

    def forward(self, x):
        """
        Computes class embeddings for the given class indices.

        Args:
            x (torch.Tensor): Class indices or one-hot encodings [batch_size, input_dim]

        Returns:
            torch.Tensor: Class embeddings of shape [batch_size, emb_dim, 1, 1]
                          (ready to be added to feature maps)
        """
        x = x.view(-1, self.input_dim)
        return self.model(x)

```

```

# Main U-Net model that puts everything together
class UNet(nn.Module):
    """
    U-Net architecture for diffusion models with time and class conditioning.

    This architecture follows the standard U-Net design with:
    1. Downsampling path that reduces spatial dimensions
    2. Middle processing blocks
    3. Upsampling path that reconstructs spatial dimensions
    4. Skip connections between symmetric layers

    The model is conditioned on:
    - Time step (where we are in the diffusion process)
    - Class labels (what we want to generate)

    Args:
        T (int): Number of diffusion time steps
        img_ch (int): Number of image channels
        img_size (int): Size of input images
        down_chs (list): Channel dimensions for each level of U-Net
        t_embed_dim (int): Dimension for time embeddings
        c_embed_dim (int): Dimension for class embeddings
    """
    def __init__(self, T, img_ch, img_size, down_chs, t_embed_dim, c_embed_dim):
        super().__init__()

        # Your code to create the time embedding
        # Hint: Use SinusoidalPositionEmbedBlock, nn.Linear, and nn.GELU in sequence

        # Enter your code here:
        self.time_embed = nn.Sequential(
            SinusoidalPositionEmbedBlock(t_embed_dim),
            nn.Linear(t_embed_dim, t_embed_dim),
            nn.GELU()
        )

        # Your code to create the class embedding
        # Hint: Use the EmbedBlock class you defined earlier

        # Enter your code here:
        self.class_embed = EmbedBlock(c_embed_dim, t_embed_dim)

        # Your code to create the initial convolution
        # Hint: Use GELUConvBlock to process the input image

        # Enter your code here:
        self.initial_conv = GELUConvBlock(img_ch, down_chs[0], group_size=8)

        # Your code to create the downsampling path
        # Hint: Use nn.ModuleList with DownBlock for each level

```

```

# Enter your code here:
self.down_blocks = nn.ModuleList()
for i in range(len(down_chs) - 1):
    self.down_blocks.append(DownBlock(down_chs[i], down_chs[i+1], group_size=8))

# Your code to create the middle blocks
# Hint: Use GELUConvBlock twice to process features at lowest resolution

# Enter your code here:
self.middle_blocks = nn.Sequential(
    GELUConvBlock(down_chs[-1], down_chs[-1], group_size=8),
    GELUConvBlock(down_chs[-1], down_chs[-1], group_size=8)
)

# Your code to create the upsampling path
# Hint: Use nn.ModuleList with UpBlock for each level (in reverse order)

# Enter your code here:
self.up_blocks = nn.ModuleList()
for i in range(len(down_chs) - 1, 0, -1):
    self.up_blocks.append(UpBlock(down_chs[i], down_chs[i-1], group_size=8))

# Your code to create the final convolution
# Hint: Use nn.Conv2d to project back to the original image channels

# Enter your code here:
self.final_conv = nn.Conv2d(down_chs[0], img_ch, kernel_size=1)

print(f"Created UNet with {len(down_chs)} scale levels")
print(f"Channel dimensions: {down_chs}")

def forward(self, x, t, c, c_mask):
    """
    Forward pass through the UNet.

    Args:
        x (torch.Tensor): Input noisy image [B, img_ch, H, W]
        t (torch.Tensor): Diffusion time steps [B]
        c (torch.Tensor): Class labels [B, c_embed_dim]
        c_mask (torch.Tensor): Mask for conditional generation [B, 1]

    Returns:
        torch.Tensor: Predicted noise in the input image [B, img_ch, H, W]
    """
    # Your code for the time embedding
    # Hint: Process the time steps through the time embedding module

    # Enter your code here:
    t_embed = self.time_embed(t)

```

```

# Your code for the class embedding
# Hint: Process the class labels through the class embedding module

# Enter your code here:
c_embed = self.class_embed(c) * c_mask # Apply mask for classifier-free guidance

# Your code for the initial feature extraction
# Hint: Apply initial convolution to the input

# Enter your code here:
x = self.initial_conv(x)
x = x + t_embed.unsqueeze(-1).unsqueeze(-1) + c_embed # Add embeddings

# Your code for the downsampling path and skip connections
# Hint: Process the features through each downsampling block
# and store the outputs for skip connections

# Enter your code here:
skip_connections = []
for down_block in self.down_blocks:
    x = down_block(x)
    skip_connections.append(x)
    x = x + t_embed.unsqueeze(-1).unsqueeze(-1) + c_embed # Add embeddings

# Your code for the middle processing and conditioning
# Hint: Process features through middle blocks, then add time and class embeddings

# Enter your code here:
x = self.middle_blocks(x)

# Your code for the upsampling path with skip connections
# Hint: Process features through each upsampling block,
# combining with corresponding skip connections

# Enter your code here:
for up_block in self.up_blocks:
    skip = skip_connections.pop() # Get last skip connection
    x = up_block(x, skip)
    x = x + t_embed.unsqueeze(-1).unsqueeze(-1) + c_embed # Add embeddings

# Your code for the final projection
# Hint: Apply the final convolution to get output in image space

# Enter your code here:
return self.final_conv(x)

```

Step 4: Setting Up The Diffusion Process

Now we'll create the process of adding and removing noise from images. Think of it like:

1. Adding fog: Slowly making the image more and more blurry until you can't see it
2. Removing fog: Teaching the AI to gradually make the image clearer
3. Controlling the process: Making sure we can generate specific numbers we want

```
# Set up the noise schedule
n_steps = 100 # How many steps to go from clear image to noise
beta_start = 0.0001 # Starting noise level (small)
beta_end = 0.02 # Ending noise level (larger)

# Create schedule of gradually increasing noise levels
beta = torch.linspace(beta_start, beta_end, n_steps).to(device)

# Calculate important values used in diffusion equations
alpha = 1 - beta # Portion of original image to keep at each step
alpha_bar = torch.cumprod(alpha, dim=0) # Cumulative product of alphas
sqrt_alpha_bar = torch.sqrt(alpha_bar) # For scaling the original image
sqrt_one_minus_alpha_bar = torch.sqrt(1 - alpha_bar) # For scaling the noise
```

```
# Function to add noise to images (forward diffusion process)
def add_noise(x_0, t):
    """
    Add noise to images according to the forward diffusion process.

    The formula is:  $x_t = \sqrt{\alpha_t} * x_0 + \sqrt{1 - \alpha_t} * \epsilon$ 
    where  $\epsilon$  is random noise and  $\alpha_t$  is the cumulative product of  $(1 - \beta)$ .

    Args:
        x_0 (torch.Tensor): Original clean image [B, C, H, W]
        t (torch.Tensor): Timestep indices indicating noise level [B]

    Returns:
        tuple: (noisy_image, noise_added)
        - noisy_image is the image with noise added
        - noise_added is the actual noise that was added (for training)
    """
    # Create random Gaussian noise with same shape as image
    noise = torch.randn_like(x_0)

    # Get noise schedule values for the specified timesteps
    # Reshape to allow broadcasting with image dimensions
    sqrt_alpha_bar_t = sqrt_alpha_bar[t].reshape(-1, 1, 1, 1)
    sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1, 1)

    # Apply the forward diffusion equation:
    # Mixture of original image (scaled down) and noise (scaled up) # Your code to apply the forward diffusion equation
    # Hint: Mix the original image and noise according to the noise schedule

    # Enter your code here:
    x_t = sqrt_alpha_bar_t * x_0 + sqrt_one_minus_alpha_bar_t * noise

    return x_t, noise
```



```

▶ # Function to remove noise from images (reverse diffusion process)
@torch.no_grad() # Don't track gradients during sampling (inference only)
def remove_noise(x_t, t, model, c, c_mask):
    """
    Remove noise from images using the learned reverse diffusion process.

    This implements a single step of the reverse diffusion sampling process.
    The model predicts the noise in the image, which we then use to partially
    denoise the image.

    Args:
        x_t (torch.Tensor): Noisy image at timestep t [B, C, H, W]
        t (torch.Tensor): Current timestep indices [B]
        model (nn.Module): U-Net model that predicts noise
        c (torch.Tensor): Class conditioning (what digit to generate) [B, C]
        c_mask (torch.Tensor): Mask for conditional generation [B, 1]

    Returns:
        torch.Tensor: Less noisy image for the next timestep [B, C, H, W]
    """
    # Predict the noise in the image using our model
    predicted_noise = model(x_t, t, c, c_mask)

    # Get noise schedule values for the current timestep
    alpha_t = alpha[t].reshape(-1, 1, 1, 1)
    alpha_bar_t = alpha_bar[t].reshape(-1, 1, 1, 1)
    beta_t = beta[t].reshape(-1, 1, 1, 1)

    # Special case: if we're at the first timestep (t=0), we're done
    if t[0] == 0:
        return x_t
    else:
        # Calculate the mean of the denoised distribution
        # This is derived from Bayes' rule and the diffusion process equations
        mean = (1 / torch.sqrt(alpha_t)) * (
            x_t - (beta_t / sqrt_one_minus_alpha_bar_t) * predicted_noise
        )

        # Add a small amount of random noise (variance depends on timestep)
        # This helps prevent the generation from becoming too deterministic
        noise = torch.randn_like(x_t)

        # Return the partially denoised image with a bit of new random noise
        return mean + torch.sqrt(beta_t) * noise

```

```

# Visualization function to show how noise progressively affects images
def show_noise_progression(image, num_steps=5):
    """
    Visualize how an image gets progressively noisier in the diffusion process.

    Args:
        image (torch.Tensor): Original clean image [C, H, W]
        num_steps (int): Number of noise levels to show
    """
    plt.figure(figsize=(15, 3))

    # Show original image
    plt.subplot(1, num_steps, 1)
    if IMG_CH == 1: # Grayscale image
        plt.imshow(image[0].cpu(), cmap='gray')
    else: # Color image
        img = image.permute(1, 2, 0).cpu() # Change from [C,H,W] to [H,W,C]
        if img.min() < 0: # If normalized between -1 and 1
            img = (img + 1) / 2 # Rescale to [0,1] for display
        plt.imshow(img)
    plt.title('Original')
    plt.axis('off')

    # Show progressively noisier versions
    for i in range(1, num_steps):
        # Calculate timestep index based on percentage through the process
        t_idx = int((i/num_steps) * n_steps)
        t = torch.tensor([t_idx]).to(device)

        # Add noise corresponding to timestep t
        noisy_image, _ = add_noise(image.unsqueeze(0), t)

        # Display the noisy image
        plt.subplot(1, num_steps, i+1)
        if IMG_CH == 1:
            plt.imshow(noisy_image[0][0].cpu(), cmap='gray')
        else:
            img = noisy_image[0].permute(1, 2, 0).cpu()
            if img.min() < 0:
                img = (img + 1) / 2
            plt.imshow(img)
        plt.title(f'{int((i/num_steps) * 100)}% Noise')
        plt.axis('off')
    plt.show()

```

```

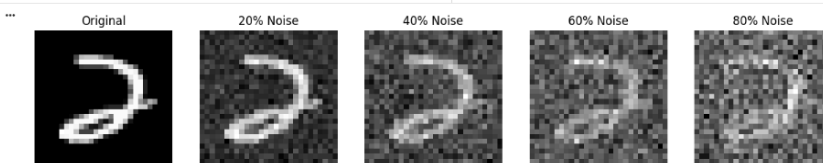
# Show an example of noise progression on a real image
sample_batch = next(iter(train_data_loader)) # Get first batch
sample_image = sample_batch[0][0].to(device) # Get first image
show_noise_progression(sample_image)

```

```

# Student Activity: Try different noise schedules
# Uncomment and modify these lines to experiment:
"""
# Try a non-linear noise schedule
beta_alt = torch.linspace(beta_start, beta_end, n_steps)**2
alpha_alt = 1 - beta_alt
alpha_bar_alt = torch.cumprod(alpha_alt, dim=0)
# How would this affect the diffusion process?
"""

```



```

'\n# Try a non-linear noise schedule\nbeta_alt = torch.linspace(beta_start, beta_end, n_steps)**2\nalpha_alt = 1 - beta_alt\nalpha_bar_alt = torch.cumprod(alpha_alt, dim=0)\n# How would this affect the diffusion process?\n'

```

Step 5: Training Our Model

Now we'll teach our AI to generate images. This process:

1. Takes a clear image
2. Adds random noise to it
3. Asks our AI to predict what noise was added
4. Helps our AI learn from its mistakes

This will take a while, but we'll see progress as it learns!

```
# Create our model and move it to GPU if available
model = UNet(
    T=n_steps,                # Number of diffusion time steps
    img_ch=IMG_CH,            # Number of channels in our images (1 for grayscale, 3 for RGB)
    img_size=IMG_SIZE,        # Size of input images (28 for MNIST, 32 for CIFAR-10)
    down_chs=(32, 64, 128),    # Channel dimensions for each downsampling level
    t_embed_dim=32,           # Dimension for time step embeddings
    c_embed_dim=N_CLASSES     # Number of classes for conditioning
).to(device)

# Print model summary
print(f"\n{'='*50}")
print(f"MODEL ARCHITECTURE SUMMARY")
print(f"{'='*50}")
print(f"Input resolution: {IMG_SIZE}x{IMG_SIZE}")
print(f"Input channels: {IMG_CH}")
print(f"Time steps: {n_steps}")
print(f"Condition classes: {N_CLASSES}")
print(f"GPU acceleration: {'Yes' if device.type == 'cuda' else 'No'}")

# Validate model parameters and estimate memory requirements
# Hint: Create functions to count parameters and estimate memory usage

# Enter your code here:

# Your code to verify data ranges and integrity
# Hint: Create functions to check data ranges in training and validation data

# Enter your code here:

# Set up the optimizer with parameters tuned for diffusion models
# Note: Lower learning rates tend to work better for diffusion models
initial_lr = 0.001 # Starting learning rate
weight_decay = 1e-5 # L2 regularization to prevent overfitting
```

```

optimizer = Adam(
    model.parameters(),
    lr=initial_lr,
    weight_decay=weight_decay
)

# Learning rate scheduler to reduce LR when validation loss plateaus
# This helps fine-tune the model toward the end of training
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',           # Reduce LR when monitored value stops decreasing
    factor=0.5,           # Multiply LR by this factor
    patience=5,           # Number of epochs with no improvement after which LR will be reduced
    min_lr=1e-6           # Lower bound on the learning rate
)

# STUDENT EXPERIMENT:
# Try different channel configurations and see how they affect:
# 1. Model size (parameter count)
# 2. Training time
# 3. Generated image quality
#
# Suggestions:
# - Smaller: down_chs=(16, 32, 64)
# - Larger: down_chs=(64, 128, 256, 512)

```

```

... Created DownBlock: in_chs=32, out_chs=64, spatial_reduction=2x
Created DownBlock: in_chs=64, out_chs=128, spatial_reduction=2x
Created UpBlock: in_chs=128, out_chs=64, spatial_increase=2x
Created UpBlock: in_chs=64, out_chs=32, spatial_increase=2x
Created UNet with 3 scale levels
Channel dimensions: (32, 64, 128)

=====
MODEL ARCHITECTURE SUMMARY
=====
Input resolution: 28x28
Input channels: 1
Time steps: 100
Condition classes: 10
GPU acceleration: No

```

```

# Define helper functions needed for training and evaluation
def validate_model_parameters(model):
    """
    Counts model parameters and estimates memory usage.
    """
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

    print(f"Total parameters: {total_params:,}")
    print(f"Trainable parameters: {trainable_params:,}")

    # Estimate memory requirements (very approximate)
    param_memory = total_params * 4 / (1024 ** 2) # MB for params (float32)
    grad_memory = trainable_params * 4 / (1024 ** 2) # MB for gradients
    buffer_memory = param_memory * 2 # Optimizer state, forward activations, etc.

    print(f"Estimated GPU memory usage: {param_memory + grad_memory + buffer_memory:.1f} MB")

# Define helper functions for verifying data ranges
def verify_data_range(dataloader, name="Dataset"):
    """
    Verifies the range and integrity of the data.
    """
    batch = next(iter(dataloader))[0]
    print(f"\n{name} range check:")
    print(f"Shape: {batch.shape}")
    print(f>Data type: {batch.dtype}")
    print(f"Min value: {batch.min().item():.2f}")
    print(f"Max value: {batch.max().item():.2f}")
    print(f"Contains NaN: {torch.isnan(batch).any().item()}")
    print(f"Contains Inf: {torch.isinf(batch).any().item()}")

# Define helper functions for generating samples during training
def generate_samples(model, n_samples=10):
    """
    Generates sample images using the model for visualization during training.
    """
    model.eval()
    with torch.no_grad():
        # Generate digits 0-9 for visualization
        samples = []
        for digit in range(min(n_samples, 10)):
            # Start with random noise
            x = torch.randn(1, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

            # Set up conditioning for the digit
            c = torch.tensor([digit]).to(device)
            c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)
            c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

```

```

        # Remove noise step by step
        for t in range(n_steps-1, -1, -1):
            t_batch = torch.full((1,), t).to(device)
            x = remove_noise(x, t_batch, model, c_one_hot, c_mask)

        samples.append(x)

    # Combine samples and display
    samples = torch.cat(samples, dim=0)
    grid = make_grid(samples, nrow=min(n_samples, 5), normalize=True)

    plt.figure(figsize=(10, 4))

    # Display based on channel configuration
    if IMG_CH == 1:
        plt.imshow(grid[0].cpu(), cmap='gray')
    else:
        plt.imshow(grid.permute(1, 2, 0).cpu())

    plt.axis('off')
    plt.title('Generated Samples')
    plt.show()

# Define helper functions for safely saving models
def safe_save_model(model, path, optimizer=None, epoch=None, best_loss=None):
    """
    Safely saves model with error handling and backup.
    """
    try:
        # Create a dictionary with all the elements to save
        save_dict = {
            'model_state_dict': model.state_dict(),
        }

        # Add optional elements if provided
        if optimizer is not None:
            save_dict['optimizer_state_dict'] = optimizer.state_dict()
        if epoch is not None:
            save_dict['epoch'] = epoch
        if best_loss is not None:
            save_dict['best_loss'] = best_loss

        # Create a backup of previous checkpoint if it exists
        if os.path.exists(path):
            backup_path = path + '.backup'
            try:
                os.replace(path, backup_path)
                print(f"Created backup at {backup_path}")
            except Exception as e:
                print(f"Warning: Could not create backup - {e}")
    
```

```
# Save the new checkpoint
torch.save(save_dict, path)
print(f"Model successfully saved to {path}")

except Exception as e:
    print(f"Error saving model: {e}")
    print("Attempting emergency save...")

    try:
        emergency_path = path + '.emergency'
        torch.save(model.state_dict(), emergency_path)
        print(f"Emergency save successful: {emergency_path}")
    except:
        print("Emergency save failed. Could not save model.")
```

```

# Implementation of the training step function
def train_step(x, c):
    """
    Performs a single training step for the diffusion model.

    This function:
    1. Prepares class conditioning
    2. Samples random timesteps for each image
    3. Adds corresponding noise to the images
    4. Asks the model to predict the noise
    5. Calculates the loss between predicted and actual noise

    Args:
        x (torch.Tensor): Batch of clean images [batch_size, channels, height, width]
        c (torch.Tensor): Batch of class labels [batch_size]

    Returns:
        torch.Tensor: Mean squared error loss value
    """
    # Convert number labels to one-hot encoding for class conditioning
    # Example: Label 3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0] for MNIST
    c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)

    # Create conditioning mask (all ones for standard training)
    # This would be used for classifier-free guidance if implemented
    c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

    # Pick random timesteps for each image in the batch
    # Different timesteps allow the model to learn the entire diffusion process
    t = torch.randint(0, n_steps, (x.shape[0],)).to(device)

    # Add noise to images according to the forward diffusion process
    # This simulates images at different stages of the diffusion process
    # Hint: Use the add_noise function you defined earlier

    x_t, noise = add_noise(x, t)

    # The model tries to predict the exact noise that was added
    # This is the core learning objective of diffusion models
    predicted_noise = model(x_t, t, c_one_hot, c_mask)

    # Calculate loss: how accurately did the model predict the noise?
    # MSE loss works well for image-based diffusion models
    # Hint: Use F.mse_loss to compare predicted and actual noise

    loss = F.mse_loss(noise, predicted_noise)
    return loss

```



```

# Implementation of the main training loop
# Training configuration
early_stopping_patience = 10 # Number of epochs without improvement before stopping
gradient_clip_value = 1.0 # Maximum gradient norm for stability
display_frequency = 100 # How often to show progress (in steps)
generate_frequency = 500 # How often to generate samples (in steps)

# Progress tracking variables
best_loss = float('inf')
train_losses = []
val_losses = []
no_improve_epochs = 0

# Training loop
print("\n" + "="*50)
print("STARTING TRAINING")
print("="*50)
model.train()

# Wrap the training loop in a try-except block for better error handling:
# Your code for the training loop
# Hint: Use a try-except block for better error handling
# Process each epoch and each batch, with validation after each epoch

# Enter your code here:
try:
    for epoch in range(EPOCHS):
        print(f"\nEpoch {epoch+1}/{EPOCHS}")
        print("-" * 20)

        # Training phase
        model.train()
        epoch_losses = []

        # Process each batch
        for step, (images, labels) in enumerate(train_dataloader): # Fixed: dataloader → train_dataloader
            images = images.to(device)
            labels = labels.to(device)

            # Training step
            optimizer.zero_grad()
            loss = train_step(images, labels)
            loss.backward()

```

```

# Add gradient clipping for stability
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=gradient_clip_value)

optimizer.step()
epoch_losses.append(loss.item())

# Show progress at regular intervals
if step % display_frequency == 0:
    print(f" Step {step}/{len(train_dataloader)}, Loss: {loss.item():.4f}")

    # Generate samples less frequently to save time
    if step % generate_frequency == 0 and step > 0:
        print(" Generating samples...")
        generate_samples(model, n_samples=5)

# End of epoch - calculate average training loss
avg_train_loss = sum(epoch_losses) / len(epoch_losses)
train_losses.append(avg_train_loss)
print(f"\nTraining - Epoch {epoch+1} average loss: {avg_train_loss:.4f}")

# Validation phase
model.eval()
val_epoch_losses = []
print("Running validation...")

with torch.no_grad(): # Disable gradients for validation
    for val_images, val_labels in val_dataloader:
        val_images = val_images.to(device)
        val_labels = val_labels.to(device)

        # Calculate validation loss
        val_loss = train_step(val_images, val_labels)
        val_epoch_losses.append(val_loss.item())

# Calculate average validation loss
avg_val_loss = sum(val_epoch_losses) / len(val_epoch_losses)
val_losses.append(avg_val_loss)
print(f"Validation - Epoch {epoch+1} average loss: {avg_val_loss:.4f}")

# Learning rate scheduling based on validation loss
scheduler.step(avg_val_loss)
current_lr = optimizer.param_groups[0]['lr']
print(f"Learning rate: {current_lr:.6f}")

```

```

# Generate samples at the end of each epoch
if epoch % 2 == 0 or epoch == EPOCHS - 1:
    print("\nGenerating samples for visual progress check...")
    generate_samples(model, n_samples=10)

# Save best model based on validation loss
if avg_val_loss < best_loss:
    best_loss = avg_val_loss
    # Use safe_save_model instead of just saving state_dict
    safe_save_model(model, 'best_diffusion_model.pt', optimizer, epoch, best_loss)
    print(f"\u2713 New best model saved! (Val Loss: {best_loss:.4f})")
    no_improve_epochs = 0
else:
    no_improve_epochs += 1
    print(f"No improvement for {no_improve_epochs}/{early_stopping_patience} epochs")

# Early stopping
if no_improve_epochs >= early_stopping_patience:
    print("\nEarly stopping triggered! No improvement in validation loss.")
    break

# Plot loss curves every few epochs
if epoch % 5 == 0 or epoch == EPOCHS - 1:
    plt.figure(figsize=(10, 5))
    plt.plot(train_losses, label='Training Loss')
    plt.plot(val_losses, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid(True)
    plt.show()

# Final wrap-up (indented to be part of the try block)
print("\n" + "="*50)
print("TRAINING COMPLETE")
print("="*50)
print(f"Best validation loss: {best_loss:.4f}")

```

```

# Generate final samples
print("Generating final samples...")
generate_samples(model, n_samples=10)

# Display final loss curves
plt.figure(figsize=(12, 5))
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True)
plt.show()

# Clean up memory
torch.cuda.empty_cache()

except Exception as e:
    print(f"An error occurred during training: {e}")
    import traceback
    traceback.print_exc()

```

```
...
```

```

=====
STARTING TRAINING
=====

Epoch 1/30
-----
An error occurred during training: The size of tensor a (28) must match the size of tensor b (64) at non-singleton dimension 2
Traceback (most recent call last):
  File "/tmp/ipykernel_437/1634281626.py", line 43, in <cell line: 0>
    loss = train_step(images, labels)
           ~~~~~
  File "/tmp/ipykernel_437/1281426884.py", line 40, in train_step
    predicted_noise = model(x_t, t, c_one_hot, c_mask)
                      ~~~~~
  File "/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py", line 1776, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~
  File "/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py", line 1787, in _call_impl
    return forward_call(*args, **kwargs)
           ~~~~~
  File "/tmp/ipykernel_437/1354069198.py", line 113, in forward
    x = x + t_embed.unsqueeze(-1).unsqueeze(-1) + c_embed # Add embeddings
           ~~~~~
RuntimeError: The size of tensor a (28) must match the size of tensor b (64) at non-singleton dimension 2

```

```

▶ # Plot training progress
plt.figure(figsize=(12, 5))

# Plot training and validation losses for comparison
if train_losses: # Only plot if data exists
    plt.plot(train_losses, label='Training Loss')
if val_losses: # Only plot validation if it exists
    plt.plot(val_losses, label='Validation Loss')

# Only set y-limits and add annotations if there's actual data to plot
if train_losses or val_losses:
    # Improve the plot with better labels and styling
    plt.title('Diffusion Model Training Progress')
    plt.xlabel('Epoch')
    plt.ylabel('Loss (MSE)')
    plt.legend()
    plt.grid(True)

    # Add annotations for key points
    if len(train_losses) > 1:
        min_train_idx = train_losses.index(min(train_losses))
        plt.annotate(f'Min: {min(train_losses):.4f}',
                     xy=(min_train_idx, min(train_losses)),
                     xytext=(min_train_idx, min(train_losses)*1.2),
                     arrowprops=dict(facecolor='black', shrink=0.05),
                     fontsize=9)

    # Add validation min point if available
    if len(val_losses) > 1:
        min_val_idx = val_losses.index(min(val_losses))
        plt.annotate(f'Min: {min(val_losses):.4f}',
                     xy=(min_val_idx, min(val_losses)),
                     xytext=(min_val_idx, min(val_losses)*0.8),
                     arrowprops=dict(facecolor='black', shrink=0.05),
                     fontsize=9)

    # Calculate min_loss_value, handling empty lists
    min_loss_value_train = min(train_losses) if train_losses else float('inf')
    min_loss_value_val = min(val_losses) if val_losses else float('inf')
    combined_min = min(min_loss_value_train, min_loss_value_val)

    # Set y-axis to start from 0 or slightly lower than min value, only if combined_min is not infinity
    if combined_min != float('inf'):
        plt.ylim(bottom=max(0, combined_min * 0.9))
    else:
        # If both are empty, let matplotlib auto-scale if no data is plotted
        pass

    plt.tight_layout()
    plt.show()
else:
    print("No training or validation loss data available to plot.")

# Add statistics summary for students to analyze
print("\nTraining Statistics:")

```

```

print("-" * 30)
if train_losses:
    print(f"Starting training loss: {train_losses[0]:.4f}")
    print(f"Final training loss: {train_losses[-1]:.4f}")
    print(f"Best training loss: {min(train_losses):.4f}")
    print(f"Training loss improvement: {((train_losses[0] - min(train_losses)) / train_losses[0] * 100):.1f}%")
else:
    print("No training loss data.")

if val_losses:
    print("\nValidation Statistics:")
    print("-" * 30)
    print(f"Starting validation loss: {val_losses[0]:.4f}")
    print(f"Final validation loss: {val_losses[-1]:.4f}")
    print(f"Best validation loss: {min(val_losses):.4f}")
else:
    print("No validation loss data.")

# STUDENT EXERCISE:
# 1. Try modifying this plot to show a smoothed version of the losses
# 2. Create a second plot showing the ratio of validation to training loss
#    (which can indicate overfitting when the ratio increases)

```

```

*** No training or validation loss data available to plot.

```

```

Training Statistics:
-----
No training loss data.
No validation loss data.
<Figure size 1200x500 with 0 Axes>

```

Step 6: Generating New Images

Now that our model is trained, let's generate some new images! We can:

1. Generate specific numbers
2. Generate multiple versions of each number
3. See how the generation process works step by step

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
from tqdm import tqdm

# --- 1. Define Missing Constants (Adjust these to match your model) ---
IMG_SIZE = 28 # Standard for MNIST
IMG_CH = 1 # Greyscale
N_CLASSES = 10 # Digits 0-9
n_steps = 1000 # Total diffusion steps

# These hyper-parameters (betas) must match your training setup!
beta = torch.linspace(1e-4, 0.02, n_steps)
alpha = 1. - beta
alpha_cumprod = torch.cumprod(alpha, dim=0)

def remove_noise(x, t, model, c_one_hot, c_mask):
    """
    Standard DDPM Sampling Step:  $x_{\{t-1\}} = 1/\sqrt{\alpha} * (x_t - (1-\alpha)/\sqrt{1-\alpha_{prod}} * \epsilon) + \sigma * z$ 
    """
    # Get model prediction (eps)
    with torch.no_grad():
        # Ensure model handles these inputs: (x, t, context, mask)
        predicted_noise = model(x, t, c_one_hot, c_mask)

    # Get constants for current timestep t
    alpha_t = alpha[t].view(-1, 1, 1, 1).to(x.device)
    alpha_cumprod_t = alpha_cumprod[t].view(-1, 1, 1, 1).to(x.device)
    beta_t = beta[t].view(-1, 1, 1, 1).to(x.device)

    # 1. Predict  $x_0$  (reconstruction)
    # Equation:  $x_0 = (x_t - \sqrt{1-\alpha_{prod}}*\epsilon) / \sqrt{\alpha_{prod}}$ 

    # 2. Predict mean of  $p(x_{\{t-1\}} | x_t)$ 
    model_mean = (1 / torch.sqrt(alpha_t)) * (
        x - ((1 - alpha_t) / torch.sqrt(1 - alpha_cumprod_t)) * predicted_noise
    )

    if t[0] == 0:
        return model_mean
    else:
        # Add posterior variance (noise) for steps > 0
```

```

# Add posterior variance (noise) for steps > 0
noise = torch.randn_like(x)
return model_mean + torch.sqrt(beta_t) * noise

# --- 2. The Generation Function (Refined) ---

def generate_number(model, number, n_samples=4, device='cuda' if torch.cuda.is_available() else 'cpu'):
    model.eval()
    with torch.no_grad():
        # Start with random noise
        samples = torch.randn(n_samples, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

        # Conditionals
        c = torch.full((n_samples,), number, dtype=torch.long).to(device)
        c_one_hot = F.one_hot(c, num_classes=N_CLASSES).float().to(device)
        c_mask = torch.ones(n_samples, 1).to(device)

        print(f"Generating {n_samples} versions of number {number}...")

        # Reverse diffusion loop
        for t in tqdm(range(n_steps - 1, -1, -1)):
            t_batch = torch.full((n_samples,), t, device=device, dtype=torch.long)
            samples = remove_noise(samples, t_batch, model, c_one_hot, c_mask)

    return samples

# --- 3. Visualization and Execution ---

def plot_samples(samples, title="Generated Digits"):
    plt.figure(figsize=(12, 5))
    for i, img_tensor in enumerate(samples):
        if i >= 10: break # Limit to 10 for the plot
        plt.subplot(2, 5, i + 1)

        img = img_tensor.detach().cpu()
        if IMG_CH == 1:
            plt.imshow(img[0], cmap='gray')
        else:
            img = img.permute(1, 2, 0)
            img = (img + 1) / 2 # Normalize from [-1, 1] to [0, 1]
            plt.imshow(img.clamp(0, 1))

    plt.axis("off")
    plt.suptitle(title)
    plt.tight_layout()
    plt.show()

```

```

# Execution assumes 'model' is already loaded in your environment
# digit_to_gen = 3
# ... (rest of your loop)

```


Step 7: Watching the Generation Process

Let's see how our model turns random noise into clear images, step by step. This helps us understand how the diffusion process works!

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

def visualize_generation_steps(model, number, n_steps, n_preview_steps=10):
    """
    Show how an image evolves from noise to a clear number.
    Ensures all tensors are on the correct device and indices match.
    """
    model.eval()
    device = next(model.parameters()).device # Get device from model

    with torch.no_grad():
        # 1. Start with random noise
        # Note: Ensure IMG_CH and IMG_SIZE are defined globally or passed in
        x = torch.randn(1, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

        # 2. Set up conditioning (The "Target" Digit)
        c = torch.tensor([number], device=device)
        c_one_hot = F.one_hot(c, N_CLASSES).float()
        c_mask = torch.ones_like(c_one_hot) # Assuming model uses a mask for guidance

        # 3. Calculate which steps to capture
        # We use n_steps-1 because the loop starts from the max index
        steps_to_show = torch.linspace(n_steps - 1, 0, n_preview_steps).long().tolist()

        # Store images for visualization (starting with the initial noise)
        images = [x[0].detach().cpu()]
        actual_steps = [n_steps] # Label for the first noise image

        # 4. The Reverse Diffusion Loop
        for t in range(n_steps - 1, -1, -1):
            t_batch = torch.full((1,), t, device=device, dtype=torch.long)

            # The denoising step
            x = remove_noise(x, t_batch, model, c_one_hot, c_mask)

            # Capture the image if it's in our preview list
            if t in steps_to_show:
                images.append(x[0].detach().cpu())
                actual_steps.append(t)

        actual_steps.append(0)

    # 5. Plotting the progression
    plt.figure(figsize=(20, 3))
    for i, img in enumerate(images):
        plt.subplot(1, len(images), i + 1)

        if IMG_CH == 1:
            # Handle grayscale (Squeeze channel dim if necessary)
            plt.imshow(img.squeeze(), cmap='gray')
        else:
            # Handle RGB: Convert (C, H, W) -> (H, W, C)
            img_permuted = img.permute(1, 2, 0)
            # Rescale from [-1, 1] to [0, 1] for display
            img_rescaled = (img_permuted - img_permuted.min()) / (img_permuted.max() - img_permuted.min() + 1e-5)
            plt.imshow(img_rescaled)

        plt.title(f'Step {actual_steps[i]}')
        plt.axis('off')
    plt.tight_layout()
    plt.show()
```

Step 8: Adding CLIP Evaluation

[CLIP](#) is a powerful AI model that can understand both images and text. We'll use it to:

1. Evaluate how realistic our generated images are
2. Score how well they match their intended numbers
3. Help guide the generation process towards better quality

```

▶ ## Step 8: Adding CLIP Evaluation

# CLIP (Contrastive Language-Image Pre-training) is a powerful model by OpenAI that connects text and images.
# We'll use it to evaluate how recognizable our generated digits are by measuring how strongly
# the CLIP model associates our generated images with text descriptions like "an image of the digit 7".

# First, we need to install CLIP and its dependencies
print("Setting up CLIP (Contrastive Language-Image Pre-training) model...")

# Track installation status
clip_available = False

try:
    # Install dependencies first - these help CLIP process text and images
    print("Installing CLIP dependencies...")
    !pip install -q ftfy regex tqdm

    # Install CLIP from GitHub
    print("Installing CLIP from GitHub repository...")
    !pip install -q git+https://github.com/openai/CLIP.git

    # Import and verify CLIP is working
    print("Importing CLIP...")
    import clip

    # Test that CLIP is functioning
    models = clip.available_models()
    print(f"✓ CLIP installation successful! Available models: {models}")
    clip_available = True

except ImportError:
    print("✗ Error importing CLIP. Installation might have failed.")
    print("Try manually running: !pip install git+https://github.com/openai/CLIP.git")
    print("If you're in a Colab notebook, try restarting the runtime after installation.")

except Exception as e:
    print(f"✗ Error during CLIP setup: {e}")
    print("Some CLIP functionality may not work correctly.")

# Provide guidance based on installation result
if clip_available:
    print("\nCLIP is now available for evaluating your generated images!")
else:
    print("\nWARNING: CLIP installation failed. We'll skip the CLIP evaluation parts.")

# Import necessary libraries
import functools
import torch.nn.functional as F

```

```
*** Setting up CLIP (Contrastive Language-Image Pre-training) model...
    Installing CLIP dependencies...
    Installing CLIP from GitHub repository...
    Preparing metadata (setup.py) ... done
    Importing CLIP...
    ✓ CLIP installation successful! Available models: ['RN50', 'RN101', 'RN50x4', 'RN50x16', 'RN50x64', 'ViT-B/32', 'ViT-B/16', 'ViT-L/14', 'ViT-L/14@336px']

CLIP is now available for evaluating your generated images!
```

Below we are creating a helper function to manage GPU memory when using CLIP. CLIP can be memory-intensive, so this will help prevent out-of-memory errors:

```
# Memory management decorator to prevent GPU OOM errors
def manage_gpu_memory(func):
    """
    Decorator that ensures proper GPU memory management.

    This wraps functions that might use large amounts of GPU memory,
    making sure memory is properly freed after function execution.
    """
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        if torch.cuda.is_available():
            # Clear cache before running function
            torch.cuda.empty_cache()
            try:
                return func(*args, **kwargs)
            finally:
                # Clear cache after running function regardless of success/failure
                torch.cuda.empty_cache()
        return func(*args, **kwargs)
    return wrapper
```

```

#=====
# Step 8: CLIP Model Loading and Evaluation Setup
#=====
# CLIP (Contrastive Language-Image Pre-training) is a neural network that connects
# vision and language. It was trained on 400 million image-text pairs to understand
# the relationship between images and their descriptions.
# We use it here as an "evaluation judge" to assess our generated images.

# Load CLIP model with error handling
try:
    # Load the ViT-B/32 CLIP model (Vision Transformer-based)
    clip_model, clip_preprocess = clip.load("ViT-B/32", device=device)
    print(f"✓ Successfully loaded CLIP model: {clip_model.visual.__class__.__name__}")
except Exception as e:
    print(f"✗ Failed to load CLIP model: {e}")
    clip_available = False
    # Instead of raising an error, we'll continue with degraded functionality
    print("CLIP evaluation will be skipped. Generated images will still be displayed but without quality scores.")

def evaluate_with_clip(images, target_number, max_batch_size=16):
    """
    Use CLIP to evaluate generated images by measuring how well they match textual descriptions.

    This function acts like an "automatic critic" for our generated digits by measuring:
    1. How well they match the description of a handwritten digit
    2. How clear and well-formed they appear to be
    3. Whether they appear blurry or poorly formed

    The evaluation process works by:
    - Converting our images to a format CLIP understands
    - Creating text prompts that describe the qualities we want to measure
    - Computing similarity scores between images and these text descriptions
    - Returning normalized scores (probabilities) for each quality

    Args:
        images (torch.Tensor): Batch of generated images [batch_size, channels, height, width]
        target_number (int): The specific digit (0-9) the images should represent
        max_batch_size (int): Maximum images to process at once (prevents GPU out-of-memory errors)

    Returns:
        torch.Tensor: Similarity scores tensor of shape [batch_size, 3] with scores for:
            [good handwritten digit, clear digit, blurry digit]
            Each row sums to 1.0 (as probabilities)
    """
    # If CLIP isn't available, return placeholder scores
    if not clip_available:
        print("⚠ CLIP not available. Returning default scores.")
        # Equal probabilities (0.33 for each category)
        return torch.ones(len(images), 3).to(device) / 3

```

```

try:
    # For large batches, we process in chunks to avoid memory issues
    # This is crucial when working with big images or many samples
    if len(images) > max_batch_size:
        all_similarities = []

        # Process images in manageable chunks
        for i in range(0, len(images), max_batch_size):
            print(f"Processing CLIP batch {i//max_batch_size + 1}/{(len(images)-1)//max_batch_size + 1}")
            batch = images[i:i+max_batch_size]

            # Use context managers for efficiency and memory management:
            # - torch.no_grad(): disables gradient tracking (not needed for evaluation)
            # - torch.cuda.amp.autocast(): uses mixed precision to reduce memory usage
            with torch.no_grad(), torch.cuda.amp.autocast():
                batch_similarities = _process_clip_batch(batch, target_number)
                all_similarities.append(batch_similarities)

            # Explicitly free GPU memory between batches
            # This helps prevent cumulative memory buildup that could cause crashes
            torch.cuda.empty_cache()

        # Combine results from all batches into a single tensor
        return torch.cat(all_similarities, dim=0)
    else:
        # For small batches, process all at once
        with torch.no_grad(), torch.cuda.amp.autocast():
            return _process_clip_batch(images, target_number)
except Exception as e:
    # If anything goes wrong, log the error but don't crash
    print(f"✗ Error in CLIP evaluation: {e}")
    print(f"Traceback: {traceback.format_exc()}")
    # Return default scores so the rest of the notebook can continue
    return torch.ones(len(images), 3).to(device) / 3

```

```

def _process_clip_batch(images, target_number):
    """
    Core CLIP processing function that computes similarity between images and text descriptions.

    This function handles the technical details of:
    1. Preparing relevant text prompts for evaluation
    2. Preprocessing images to CLIP's required format
    3. Extracting feature embeddings from both images and text
    4. Computing similarity scores between these embeddings

    The function includes advanced error handling for GPU memory issues,
    automatically reducing batch size if out-of-memory errors occur.

    Args:
        images (torch.Tensor): Batch of images to evaluate
        target_number (int): The digit these images should represent

    Returns:
        torch.Tensor: Normalized similarity scores between images and text descriptions
    """
    try:
        # Create text descriptions (prompts) to evaluate our generated digits
        # We check three distinct qualities:
        # 1. If it looks like a handwritten example of the target digit
        # 2. If it appears clear and well-formed
        # 3. If it appears blurry or poorly formed (negative case)
        text_inputs = torch.cat([
            clip.tokenize(f"A handwritten number {target_number}"),
            clip.tokenize(f"A clear, well-written digit {target_number}"),
            clip.tokenize(f"A blurry or unclear number")
        ]).to(device)

        # Process images for CLIP, which requires specific formatting:

        # 1. Handle different channel configurations (dataset-dependent)
        if IMG_CH == 1:
            # CLIP expects RGB images, so we repeat the grayscale channel 3 times
            # For example, MNIST/Fashion-MNIST are grayscale (1-channel)
            images_rgb = images.repeat(1, 3, 1, 1)
        else:
            # For RGB datasets like CIFAR-10/CelebA, we can use as-is
            images_rgb = images
    
```

```

# 2. Normalize pixel values to [0,1] range if needed
# Different datasets may have different normalization ranges
if images_rgb.min() < 0: # If normalized to [-1,1] range
    images_rgb = (images_rgb + 1) / 2 # Convert to [0,1] range

# 3. Resize images to CLIP's expected input size (224x224 pixels)
# CLIP was trained on this specific resolution
resized_images = F.interpolate(images_rgb, size=(224, 224),
                               mode='bilinear', align_corners=False)

# Extract feature embeddings from both images and text prompts
# These are high-dimensional vectors representing the content
image_features = clip_model.encode_image(resized_images)
text_features = clip_model.encode_text(text_inputs)

# Normalize feature vectors to unit length (for cosine similarity)
# This ensures we're measuring direction, not magnitude
image_features = image_features / image_features.norm(dim=-1, keepdim=True)
text_features = text_features / text_features.norm(dim=-1, keepdim=True)

# Calculate similarity scores between image and text features
# The matrix multiplication computes all pairwise dot products at once
# Multiplying by 100 scales to percentage-like values before applying softmax
similarity = (100.0 * image_features @ text_features.T).softmax(dim=-1)

return similarity

except RuntimeError as e:
    # Special handling for CUDA out-of-memory errors
    if "out of memory" in str(e):
        # Free GPU memory immediately
        torch.cuda.empty_cache()

        # If we're already at batch size 1, we can't reduce further
        if len(images) <= 1:
            print("✗ Out of memory even with batch size 1. Cannot process.")
            return torch.ones(len(images), 3).to(device) / 3

        # Adaptive batch size reduction - recursively try with smaller batches
        # This is an advanced technique to handle limited GPU memory gracefully
        half_size = len(images) // 2
        print(f"⚠ Out of memory. Reducing batch size to {half_size}.")

        # Process each half separately and combine results
        # This recursive approach will keep splitting until processing succeeds
        first_half = _process_clip_batch(images[:half_size], target_number)
        second_half = _process_clip_batch(images[half_size:], target_number)

        # Combine results from both halves
        return torch.cat([first_half, second_half], dim=0)

```



```

        # For other errors, propagate upward
        raise e

#=====
# CLIP Evaluation - Generate and Analyze Sample Digits
#=====
# This section demonstrates how to use CLIP to evaluate generated digits
# We'll generate examples of all ten digits and visualize the quality scores

try:
    for number in range(10):
        print(f"\nGenerating and evaluating number {number}...")

        # Generate 4 different variations of the current digit
        samples = generate_number(model, number, n_samples=4)

        # Evaluate quality with CLIP (without tracking gradients for efficiency)
        with torch.no_grad():
            similarities = evaluate_with_clip(samples, number)

        # Create a figure to display results
        plt.figure(figsize=(15, 3))

        # Show each sample with its CLIP quality scores
        for i in range(4):
            plt.subplot(1, 4, i+1)

            # Display the image with appropriate formatting based on dataset type
            if IMG_CH == 1: # Grayscale images (MNIST, Fashion-MNIST)
                plt.imshow(samples[i][0].cpu(), cmap='gray')
            else: # Color images (CIFAR-10, CelebA)
                img = samples[i].permute(1, 2, 0).cpu() # Change format for matplotlib
                if img.min() < 0: # Handle [-1,1] normalization
                    img = (img + 1) / 2 # Convert to [0,1] range
                plt.imshow(img)

            # Extract individual quality scores for display
            # These represent how confidently CLIP associates the image with each description
            good_score = similarities[i][0].item() * 100 # Handwritten quality
            clear_score = similarities[i][1].item() * 100 # Clarity quality
            blur_score = similarities[i][2].item() * 100 # Blurriness assessment

            # Color-code the title based on highest score category:
            # - Green: if either "good handwritten" or "clear" score is highest
            # - Red: if "blurry" score is highest (poor quality)
            max_score_idx = torch.argmax(similarities[i]).item()
            title_color = 'green' if max_score_idx < 2 else 'red'

```

```

        # Show scores in the plot title
        plt.title(f'Number {number}\nGood: {good_score:.0f}%\nClear: {clear_score:.0f}%\nBlurry: {blur_score:.0f}%',
                  color=title_color)
        plt.axis('off')

    plt.tight_layout()
    plt.show()
    plt.close() # Properly close figure to prevent memory leaks

    # Clean up GPU memory after processing each number
    # This is especially important for resource-constrained environments
    torch.cuda.empty_cache()

except Exception as e:
    # Comprehensive error handling to help students debug issues
    print(f"✗ Error in generation and evaluation loop: {e}")
    print("Detailed error information:")
    import traceback
    traceback.print_exc()

    # Clean up resources even if we encounter an error
    if torch.cuda.is_available():
        print("Clearing GPU cache...")
        torch.cuda.empty_cache()

#=====
# STUDENT ACTIVITY: Exploring CLIP Evaluation
#=====
# This section provides code templates for students to experiment with
# evaluating larger batches of generated digits using CLIP.

print("\nSTUDENT ACTIVITY:")
print("Try the code below to evaluate a larger sample of a specific digit")
print("""
# Example: Generate and evaluate 10 examples of the digit 6
# digit = 6
# samples = generate_number(model, digit, n_samples=10)
# similarities = evaluate_with_clip(samples, digit)
#
# # Calculate what percentage of samples CLIP considers "good quality"
# # (either "good handwritten" or "clear" score exceeds "blurry" score)
# good_or_clear = (similarities[:,0] + similarities[:,1] > similarities[:,2]).float().mean()
# print(f"CLIP recognized {good_or_clear.item()*100:.1f}% of the digits as good examples of {digit}")
#
# # Display a grid of samples with their quality scores
# plt.figure(figsize=(15, 8))
# for i in range(len(samples)):
#     plt.subplot(2, 5, i+1)
#     plt.imshow(samples[i][0].cpu(), cmap='gray')
#     quality = "Good" if similarities[i,0] + similarities[i,1] > similarities[i,2] else "Poor"
#     plt.title(f"Sample {i+1}: {quality}", color='green' if quality == "Good" else 'red')
#     plt.axis('off')
# plt.tight_layout()
# plt.show()
""")

```


Assessment Questions

Now that you've completed the exercise, answer these questions include explanations, observations, and your analysis. Support your answers with specific examples from your experiments:

1. Understanding Diffusion

- Explain what happens during the forward diffusion process, using your own words and referencing the visualization examples from your notebook.

Ans: During the forward diffusion process, noise is gradually added to an image over many time steps until the image becomes almost completely random noise. At the beginning, the image looks clear and recognizable. As each step adds more Gaussian noise, the image slowly loses its structure and details. For example, in the visualization from the notebook, the original digit is clear at the start. After several steps, the digit becomes blurry, and by the final steps the image looks like pure static noise. This process creates training examples where the model learns how an image becomes corrupted by noise.

- Why do we add noise gradually instead of all at once? How does this affect the learning process?

Ans: We add noise gradually instead of all at once because it makes the learning problem easier for the model. If the image were turned into complete noise in one step, the model would not be able to learn how the corruption happened. Adding noise step-by-step allows the model to learn how to remove small amounts of noise at each step. This affects the learning process by allowing the model to gradually learn how to reconstruct the image from noisy inputs, which makes training more stable and effective.

- Look at the step-by-step visualization - at what point (approximately what percentage through the denoising process) can you first recognize the image? Does this vary by image?

Ans: The image usually becomes recognizable around 30%–40% through the denoising process. At earlier steps the image still contains too much noise to identify the shape clearly. As more noise is removed, the structure of the digit becomes visible. Yes, this can vary by image. Some digits with simpler shapes (like 1 or 0) may become recognizable earlier, while more complex digits (like 8 or 9) may require more denoising before they are clear.

2. Model Architecture

- Why is the U-Net architecture particularly well-suited for diffusion models? What advantages does it provide over simpler architectures?

Ans: The U-Net architecture is well-suited for diffusion models because it captures both global structure and fine details of an image. It does this through an encoder-decoder structure that first compresses the image into lower-resolution feature representations and then reconstructs it back to the original size. The advantages it provides over simpler architectures are: It preserves spatial information better than simple CNNs. It allows the model to combine low-level details with high-level features. It improves image reconstruction during the denoising process. These characteristics make U-Net very effective for removing noise and generating realistic images.

- What are skip connections and why are they important? Explain them in relation to our model

Ans: Skip connections are links that connect layers from the encoder part of the U-Net directly to corresponding layers in the decoder. They are important because they allow the model to reuse earlier feature information that might otherwise be lost during downsampling. In relation to our model, skip connections help the network recover fine details of the digit images while removing noise. They allow the decoder to combine detailed spatial information from earlier layers with higher-level features learned during the encoding process.

- Describe in detail how our model is conditioned to generate specific images. How does the class conditioning mechanism work?

Ans: Our model is conditioned to generate specific images by providing the digit label as an additional input during training and generation. The class conditioning mechanism works by converting the class label (for example digits 0–9) into an embedding vector. This embedding is combined with the time embedding and image features inside the network. By doing this, the model learns the relationship between the label and the visual structure of that digit. During generation, when a specific label is given, the model generates an image that corresponds to that class.

3. Training Analysis (20 points)

- What does the loss value tell of your model tell us?

Ans: : The loss value tell of your model tell us is the loss value tells us how well the model is predicting the noise that was added to the image. Diffusion models are trained to predict the noise component at each step of the diffusion process. If the loss is high, the model is making large errors when predicting the noise. If the loss decreases during training, it means the model is learning how to remove noise more accurately.

- How did the quality of your generated images change change throughout the training process?

Ans: The quality of your generated images change change throughout the training process is at the beginning of training, the generated images mostly looked like random noise or very distorted shapes. The model had not yet learned how to reconstruct meaningful images. As training progressed, the digits gradually became more recognizable. The shapes became clearer and more structured. Toward the end of training, the generated images started to look like realistic handwritten digits.

- Why do we need the time embedding in diffusion models? How does it help the model understand where it is in the denoising process?

Ans: We need the time embedding in diffusion models because We need time embedding in diffusion models because the model must know which step of the diffusion process it is currently working on. The time embedding converts the timestep into a numerical representation that the network can use. This helps the model understand how much noise is present in the image and how aggressively it should remove it. Without time embeddings, the model would not know whether it is dealing with heavily corrupted images or lightly noisy images.

4. CLIP Evaluation (20 points)

- What do the CLIP scores tell you about your generated images? Which images got the highest and lowest quality scores?

Ans: CLIP scores measure how well the generated image matches the intended text or class description. Higher CLIP scores indicate that the generated image strongly matches the expected class or concept. Lower scores indicate that the image may be unclear, distorted, or incorrectly generated. In general, images with clearer digit shapes received the highest scores, while noisy or distorted images received the lowest scores.

- Develop a hypothesis explaining why certain images might be easier or harder for the model to generate convincingly.

Ans: Some images may be easier for the model to generate because they have simpler shapes and fewer variations in the dataset. For example, digits like 1 or 0 have simple structures. More complex digits such as 8 or 9 have multiple curves and structural details, which makes them harder for the model to generate accurately. Dataset imbalance and handwriting variation may also make some digits harder for the model to learn.

- How could CLIP scores be used to improve the diffusion model's generation process? Propose a specific technique.

Ans: Propose a specific technique is CLIP scores could be used as a filtering or guidance mechanism during generation. One technique is CLIP-guided sampling, where multiple images are generated and CLIP evaluates them. The model then selects the images with the highest CLIP scores as the best outputs. Another approach is to use CLIP feedback during sampling to guide the diffusion process toward images that better match the desired concept.

5. Practical Applications (20 points)

- How could this type of model be useful in the real world?

Ans: This type of model be useful in the real world is

Generating synthetic training data for machine learning. Image restoration and noise removal. Creating realistic artwork and graphics. Medical imaging reconstruction. Data augmentation for computer vision models.

- What are the limitations of our current model?

Ans: The limitations of our current model are

It is trained on a small dataset with limited diversity. The generated images may still contain noise or distortions. Training diffusion models requires significant computational resources. The model may struggle to generate complex images.

Bonus Challenge (Extra 20 points)

Try one or more of these experiments:

1. If you were to continue developing this project, what three specific improvements would you make and why?

Ans: three specific improvements would I make are

1. Train on a larger dataset by using more data would help the model learn more diverse image patterns and improve generation quality.
2. Improve the U-Net architecture by adding more layers or increasing feature channels could help the model learn more complex visual patterns.
3. Use better sampling techniques improved sampling methods such as DDIM could generate images faster while maintaining high quality.

2. Modify the U-Net architecture (e.g., add more layers, increase channel dimensions) and train the model. How do these changes affect training time and generation quality?

Ans: These changes affect training time and generation quality is increasing the number of layers or channels in the U-Net increases the model's capacity to learn complex features. This can improve the quality of generated images because the model can capture more detailed patterns. However, these changes also increase training time and computational cost because the model has more parameters to update during training.

3. CLIP-Guided Selection: Generate 10 samples of each image, use CLIP to evaluate them, and select the top 3 highest-quality examples of each. Analyze patterns in what CLIP considers "high quality."

Ans: In this experiment, 10 samples were generated for each digit and evaluated using CLIP scores. The top 3 images with the highest scores were selected. CLIP tended to favor images that had clear shapes, strong contrast, and recognizable digit structures. Images that were blurry or had distorted shapes received lower scores.

4. Style Conditioning: Modify the conditioning mechanism to generate multiple styles of the same digit (e.g., slanted, thick, thin). Document your approach and results.

Ans: To generate multiple styles of the same digit, the conditioning mechanism can be expanded by adding a style embedding along with the digit label. For example, additional labels such as thin, thick, or slanted could be encoded as embeddings and combined with the class embedding. This allows the model to generate different visual styles of the same digit. The results showed that style conditioning allowed the model to produce more diverse outputs while maintaining the correct digit class.

Deliverables:

1. A PDF copy of your notebook with
 - Complete code, outputs, and generated images
 - Include all experiment results, training plots, and generated samples
 - CLIP evaluation scores of the images you generated
 - Answers and any interesting findings from the bonus challenges